

PROGRAMMING DESIGN SYSTEMS

INTRO FORM 1 FORM 2 COLOR TYPOGRAPHY GRID LOGO RANDOMIZATION REPETITION TRANSFORMATION MOTION 3D EXAMPLES

COMPUTATIONAL COLOR

If you expect to come out of this class knowing simple rules like “red is for love” or “green is for happiness”, you’ll be extremely disappointed today.

Color is way too subjective and dynamic to allow for so simple definitions. Actually, I’d rather stay away from telling you which colors to use when. Instead we will do 2 things:

- Learn to, simply, observe color and its characteristics.
- Learn how the ideas of color combination springs from simple rules performed on the color spectrum



The Woman in the Red Dress - Full Scene - MATRIX - [4K]
Cringe Cook



Watch on

The woman in the red dress is a character in a training program built by the rebels in the Matrix. More specifically, [Mouse](#) designed her... and he picked the color red for her dress. Although many find it hard to work creatively with color, very few people would question why he chose this. It’s attractive, erotic, wild and draws attention.

There are 2 reasons why I choose to show this scene:

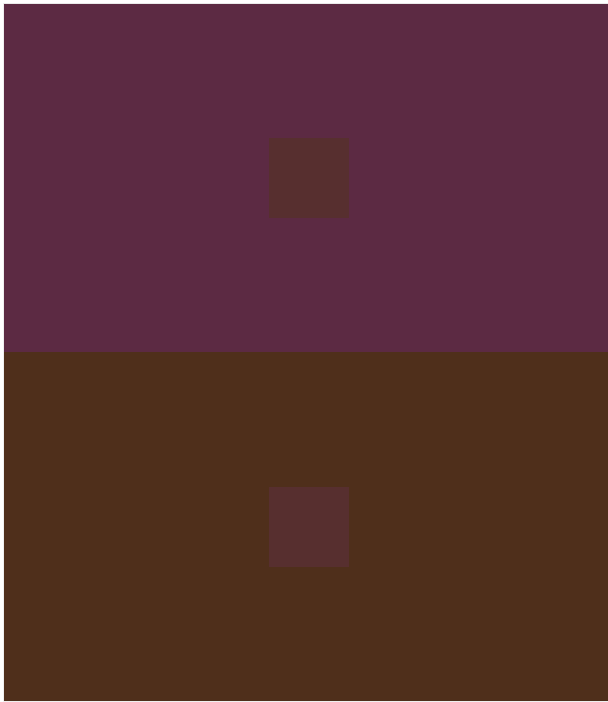
1. The use of color in this movie is fantastic. A vague green color is used for all scenes in the Matrix. The slaves in the Matrix are dressed in gray. Scenes in the Nebuchadnezzar are tinted blue to symbolize clouds, heaven and light - the real world. Everything has a symbolic meaning, and color is used to emphasize it.
2. Once you learn to control color, you’ll be like Neo moving through the Matrix. You’ll see things that no one else sees.

Color is one the hardest things to master in the graphic arts, as it’s both very subjective and highly dynamic (our perception of a single color change depending on what other colors we pair it with).

Here’s a quick example of how dynamic behavior color combination can have. This looks like 2 grounds where the grounds exchanged in the center squares. They are - in fact - 3 colors, where the center square is a color midways in the hue spectrum between the grounds. The eye is searching for relevance.

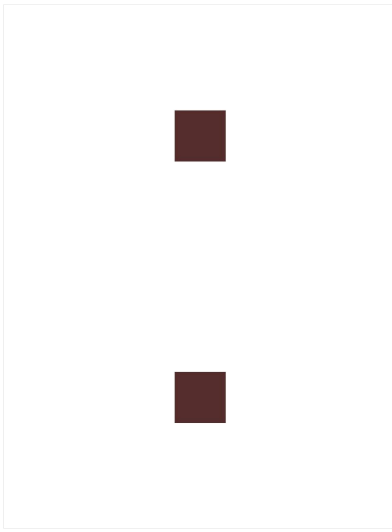
On this page

- What is Color?
- Subtractive vs. Additive Color Systems
- What’s wrong with RGB?
- HSV And Color modes in Rune.js
- Rune.Color
- Generating Monochromatic Color Schemes
- Generating Analogous Color Schemes
- Generating Complementary Color Schemes
- Generating Triadic Color Schemes
- Generating Tetradic Color Schemes
- Itten’s Color Contrasts
- Contrast of Light and Dark
- Contrast of Saturation
- Contrast of Hue
- Cold / Warm Contrast
- Contrast of Extension
- Generating colors
- How to Find Opposite Brightness
- Go to slideshow



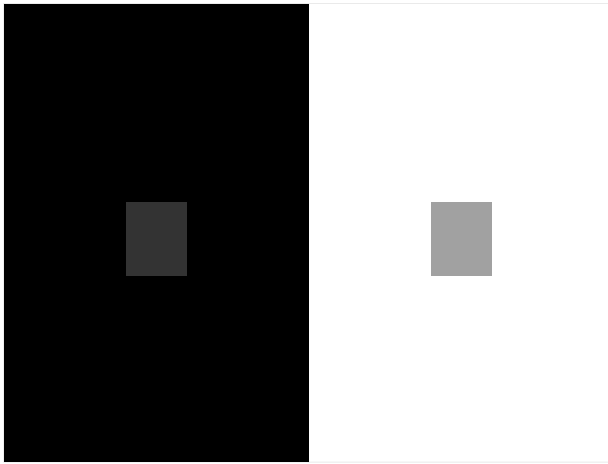
[See example code](#)

We can prove this by removing the grounds:



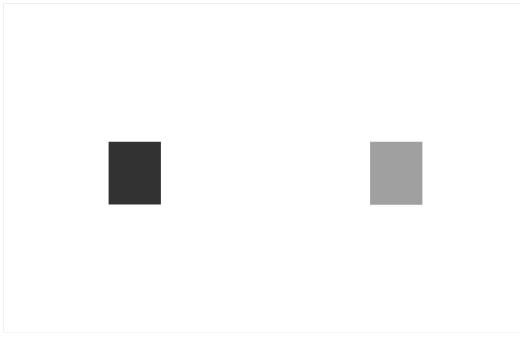
The conclusion to this must be that color theory matters: Based on simple calculation we created an effect in the eye of the beholder.

The problem with combining color only by numbers is that you can start to trust the numbers instead of your eyes. Here's another example of why this is problematic. In this example the small squares looks like the exact same color, but they are not even close. Because they keep a relative distance in brightness to their respective grounds, they look the same to the eye. We will investigate this further later when talking about simultaneous contrast.



[See example code](#)

Again, let's remove the background and see.



What is Color?

When you direct white light through a prism, you get a range of colors. White light holds all colors. When you mix all colors together, you get white. If you remove all colors, you get black.



This is why the sun turns red when setting. The atmosphere is a filter that removes colors.



Color is all around us. Color express meaning. It's a natural way of communication.



Subtractive vs. Additive Color Systems

Let's say that you walk down to Blick and buy a beginner's painting set and mix all of the colors together in a bucket. What color will you get? - Black.

Let's say you then open a new processing sketch and mixes all colors together. What color do you get? - White.

Obviously there are 2 different ways that colors mix: by addition and by subtraction.

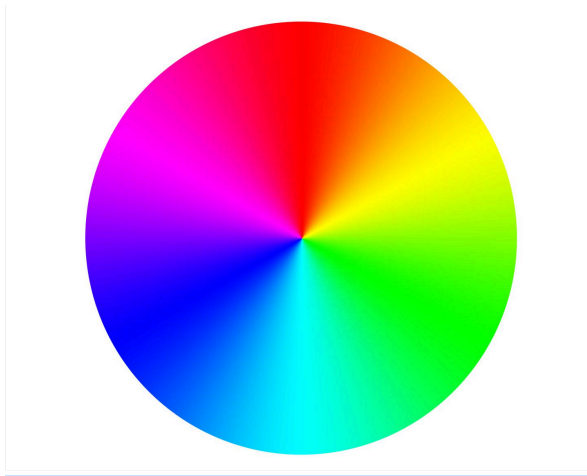
This is Johannes Itten's color wheel. It's built on Goethe's color wheel, and this is the wheel that most of you are familiar with. The RYB wheel.



All colors created with paint or ink will combine in a **subtractive** way, meaning that if you mix all colors together, the resulting color will be black (pigments work by absorbing light and reflecting the remaining colors).

If you use RGB as the primary colors when painting, you cannot mix yellow. This is why RYB was chosen as primary colors, and this had a tremendous impact on color use in art.

Then there's the RGB wheel:



All colors created with light will combine in an **additive** way, meaning that if you mix all colors together, the resulting color will be white. RGB is just one model that simulates this.

The truth is that color is a sliding spectrum, and color wheels are just basic ways of dividing this spectrum. Actually, you can use any three colors to define what's called [a color gamut](#), a subset of all possible colors.

So when we talk about things like "complementary" or "opposite" color, this varies depending on your color wheel. The complementary color of "red" in the RYB color wheel is "green", but "cyan" in the RGB wheel.

Because we are working in both worlds, we are faced with a problem: we are designing on a screen that uses additive blending, but printing on a printer that uses subtractive blending. AMS solved this problem for us because the printers are optimized for the Adobe RGB 1998 color profile.

What's wrong with RGB?

Most of you are probably totally comfortable using the RGB color mode. If we want to make a blue color, we do this:

```
.fill(0, 0, 255); // this is blue
```

And if we want to draw a green color we do this:

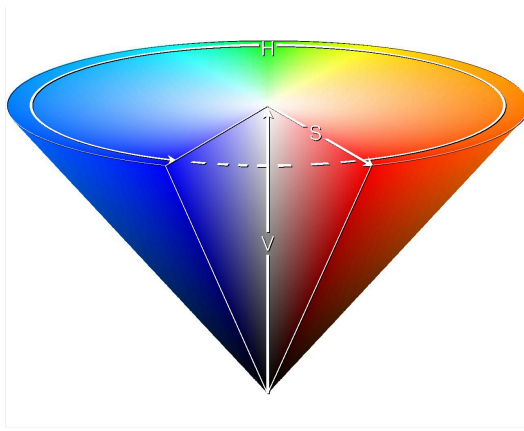
```
.fill(0, 255, 0); // this is green
```

But once you start doing generative color, you realize that RGB is not a good way of working. For example, here's a color.



It's really hard to move around the color spectrum in RGB.

HSB is a color model that exists as a 3D color space, where hue, saturation and brightness has an axis each.

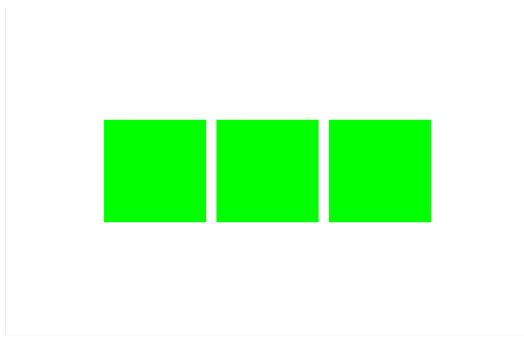


This makes it much easier to move around in on the color wheel, because one of those axis is hue, the actual hue of the color.

HSV And Color modes in Rune.js

By default, all color functions in Rune.js use the RGB color mode, expecting values from 0 to 255. However, by passing in “hsv” as the first argument, you can change the default color mode. The following example demonstrates how to draw the same green rectangle in both RGB and HSV.

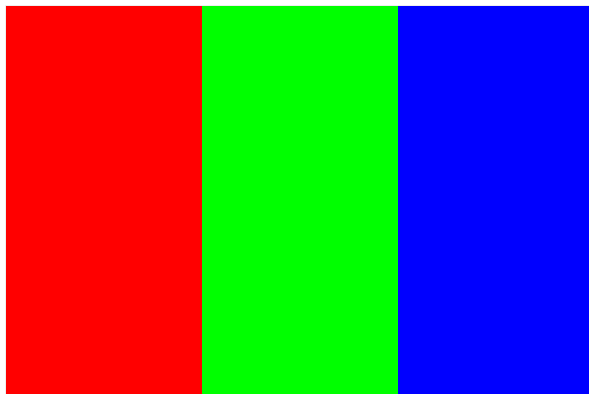
Here's a example that demonstrate how to draw the color green in 3 different color modes.



[Example on Github](#)

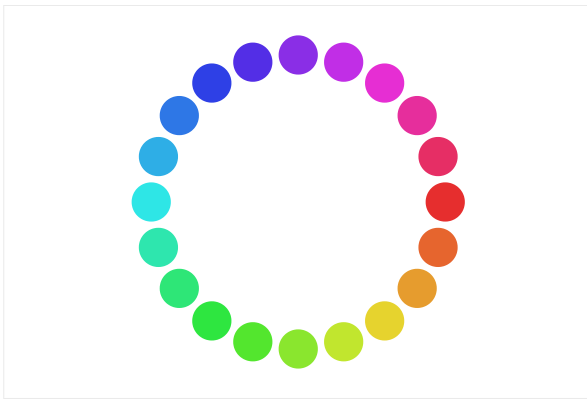
HSV defaults to hue values that range from 0-360, because that's the number of degrees in a full circle. This is also the default in Photoshop.

The first steps into working with HSB is to just draw simple colors on the screen. Here's a sketch that shows how to draw a red, green and blue square, along with some other colors.



[See example code](#)

Now that we know how to move around the hue wheel, we can use this technique to draw a color circle.



[See example code](#)

Rune.Color

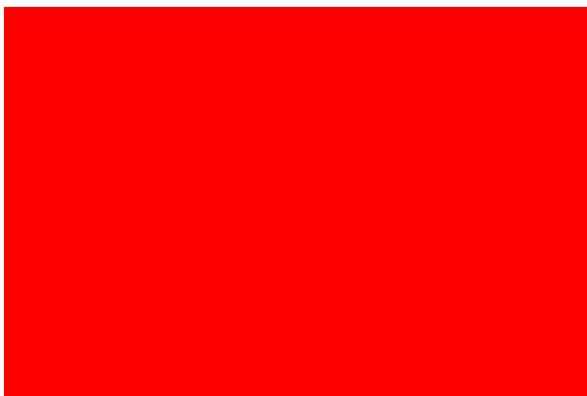
See [Using colors](#) in the Rune.js documentation. Also see the [color functions](#) example.

Generating Monochromatic Color Schemes

The different ways of creating color schemes are basically just constraints to help you not float around randomly in the 3D color space. All these examples use TColor.



One of the simplest possible ways of generating a color scheme is to pick a specific hue and saturation, and then choose colors that are evenly distributed on the brightness scale. This is one example of a **monochromatic color scheme**.



[See example code](#)

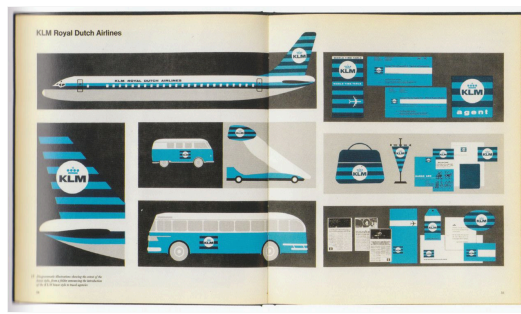
This sketch has an even distribution in brightness values. You can easily imagine code that choose the values randomly.

A slightly more sophisticated **monochromatic color scheme** would be to manipulate both saturation and brightness:

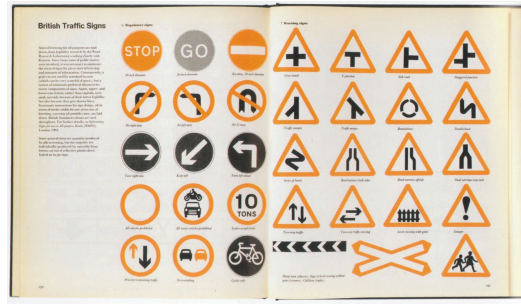
[Color Scheme Saturation Brightness](#)

[See example code](#)

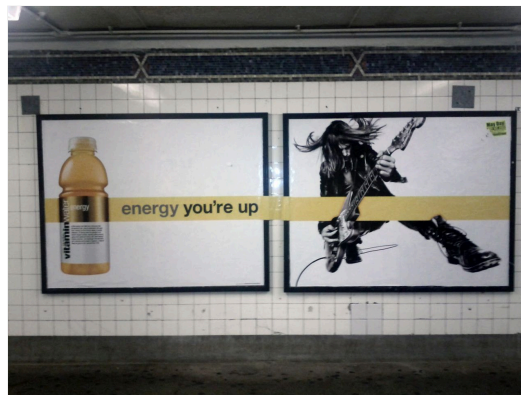
Monochromatic can be used for very simple and minimalistic graphics. Often you see it used with pure black or white to create contrast.



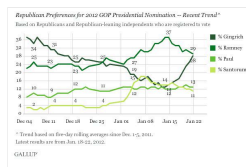
This is an example of E.H.K. Herion's design for KLM Royal Dutch Airlines with a monochrome color profile.



Traffic signs, here British traffic signs from the 1960's, generally have a monochromatic color profile, in order not to confuse the driver. Color is selectively used for drawing attention to important instructions.



We are hip. We are trendy. We use monochromatic color.



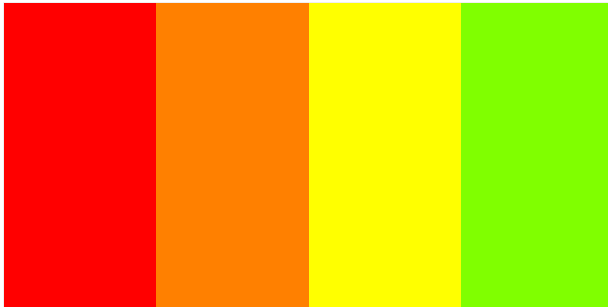
Here's an example of extremely bad use of a monochromatic color scheme. The only role of the color in this graph is to connect names with lines. How can we do that when the colors are almost identical?

Generating Analogous Color Schemes



And analogous color scheme is a suit of colors that are located close to each other on the color wheel. Often this is a color located 30 degrees away from the base color hue on either side, but that degree can vary.

We rotate 30 degrees around the color wheel because the standard color wheel has 12 colors, and $360/12$ is 30. We are just finding the “next” color on the wheel.



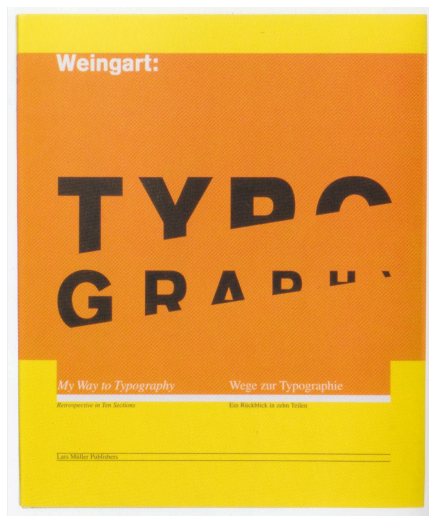
[See example code](#)



Analogous color schemes are found in nature and are often very harmonious to the eye.



With the return of the letterpress greeting card in the 2000's, I've noticed an extreme use of analogous color schemes.



Here's an example of analogous color use from Wolfgang Weingart's book about typography.

Generating Complementary Color Schemes

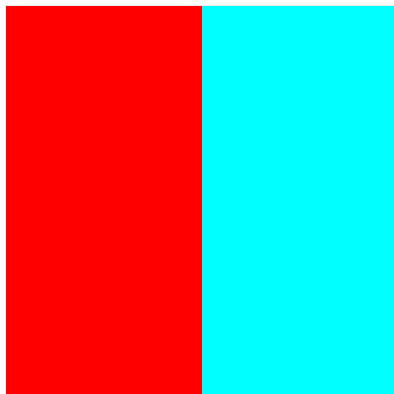
A complementary color is a color that exist on the opposite side of the color wheel. Older color theorists considered this to be the definition of color harmony, as complementary colors will always mix to grayish black (in paint) or grayish white (in light).

Some complementary colors are: Red = Cyan, Green = Magenta, Blue = Yellow.



Look at a green square for a while, and then close your eyes. You will now see a cyan square. The eye tends to look for balance: tried to find colors that mix to white.

It's easy to find a complementary color in Rune.js. Notice the sharp contrast between the two colors.



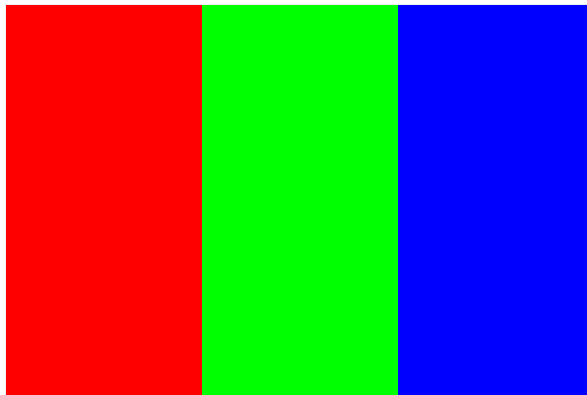
[See example code](#)

Generating Triadic Color Schemes



A triadic color can be said to be the opposite of an analogous color. Where analogous colors are colors located within 90 degrees of the primary color hue, triadic colors are colors located more than 90 degrees away of the base color hue.

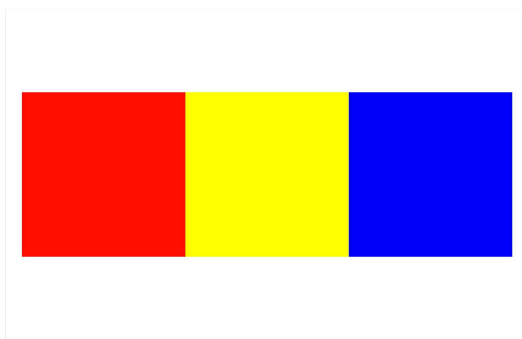
Here's a "pure" triadic color scheme with colors found by rotating 120 degrees on each side (internally an Equilateral triangle with 60 degree internal angles).



[See example code](#)

You may notice how this is the primary colors, which makes sense given the calculation.

An interesting thing is to compare a triadic color scheme in RGB with the corresponding scheme in a subtractive color mode like RYB.



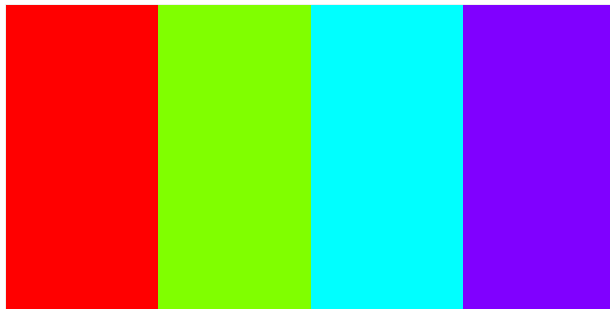
You may notice how the blue/yellow has a triadic relationship in RGB, but are complementary colors in RYB.

Generating Tetradic Color Schemes



A tetradic color scheme is a set of 4 colors chosen by placing a rectangle on the color wheel and selecting the colors that fall on the corner of the rectangle.

This color scheme is basically the same as a double complementary color scheme. Here's an example of color chosen by rotating 90 degrees around the color wheel 3 times.



[See example code](#)

Notice the cold/warm contrast in this color scheme.

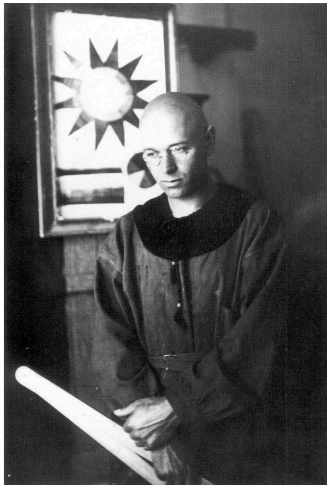
However, the rectangle does not need to be equal-sided. You could use the same base complementary pair, but with a smaller rectangle width used to pick the other complementary pair.

This increases the cold/warm contrast.



Those are the basic color scheme variations you can play with. This brings us to Itten's color contrasts.

Itten's Color Contrasts



Johannes Itten Proposed 7 different contrasts that you should be looking for when using color. The idea is to be skilled in the color scale, like you can be skilled in the tonal scale:

If one is not able to distinguish the difference between a higher tone and a lower tone, one probably should not make music - Josef Albers

Let's go through a few of them.

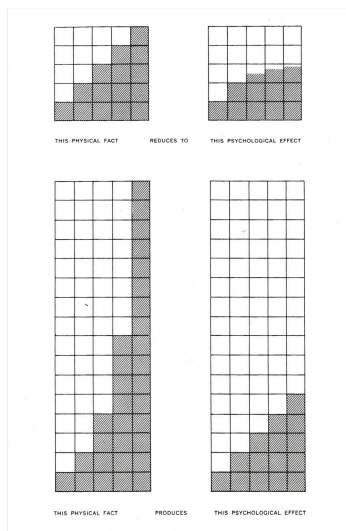
Contrast of Light and Dark

The contrast of light and dark is very simple to understand: whenever we have two colors with a difference in brightness, a clear contrast between the two colors will exist.

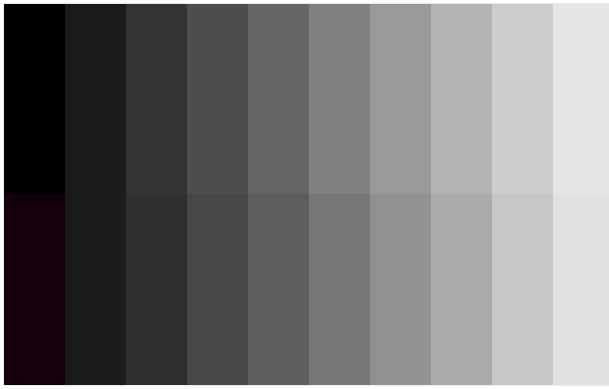
The HSB color mode makes it very obvious what kind of brightness we are working with.

However computational light/dark scales are not linear. One might think that you can make a scale of equally contrasted rectangles by moving from brightness 10 to brightness 90 in steps of 10.

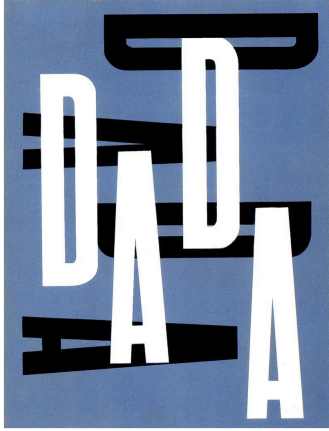
However, as first discovered in the so-called Weber-Fechner Law, a linear increase in brightness will distribute the contrast visually exponentially from white to black. An exponential increase in brightness will distribute the contrast linearly across all segments.



The following example shows both a linear increase and an exponential increase in contrast. Notice how the latter is much more even. Unfortunately this formula won't work in low- or high-brightness colors, but it's a good general rule.



[See example code](#)



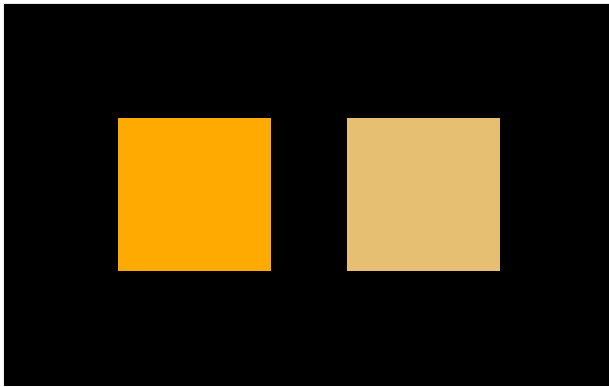
Front cover designed by Paul Rand. Notice the light-dark contrast and use of monochromatic color scheme.



Another example of light-dark contrast by Paul Rand.

Contrast of Saturation

The contrast of saturation describes the contrast that a brilliant color has towards a dull color of the same hue.



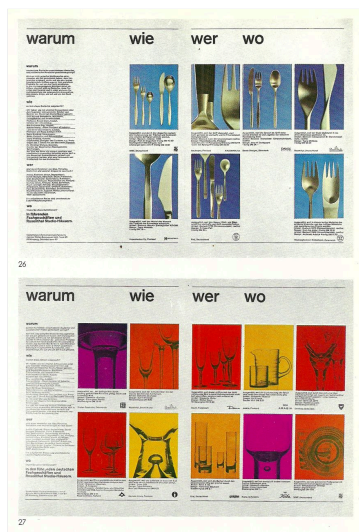
[See example code](#)

This is an example of 2 colors with contrast in saturation.



Here's an example of saturation contrast in the painting "Fish Magic" by Paul Klee.

Although there is also a light/dark contrast, it's specifically the brilliance of the fish contrasted to the dull background that makes it come alive.



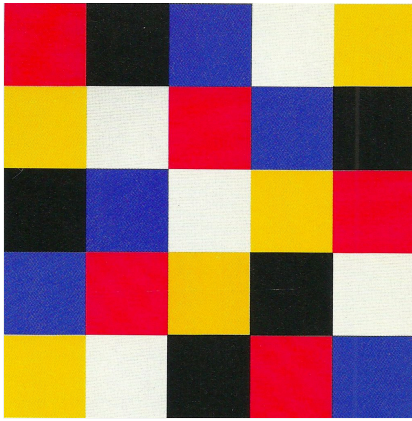
Josef Muller-Brockmann uses saturated color boxes in contrast with the gray background.

Contrast of Hue

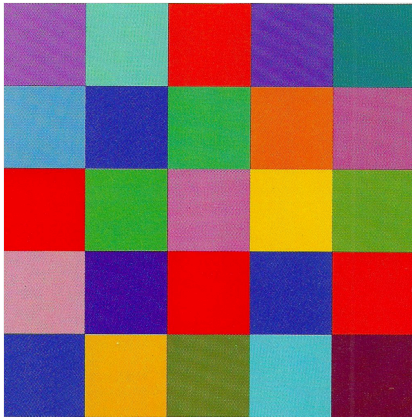
The contrast of hue describes the contrast between different color hues. The bigger the distance in hue, the bigger the contrast.

A pure triadic color scheme is the extreme instance of hue contrast.

Contrast of hue is extremely expressive, especially in brilliant colors.



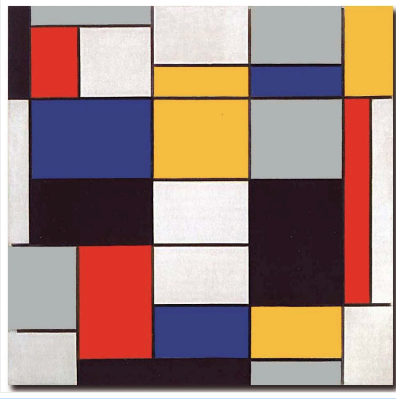
Here's an example of a strong contrast of hue, using full saturated colors and black+white.



Removing black+white and adding colors closer to each other diminishes the hue contrast.



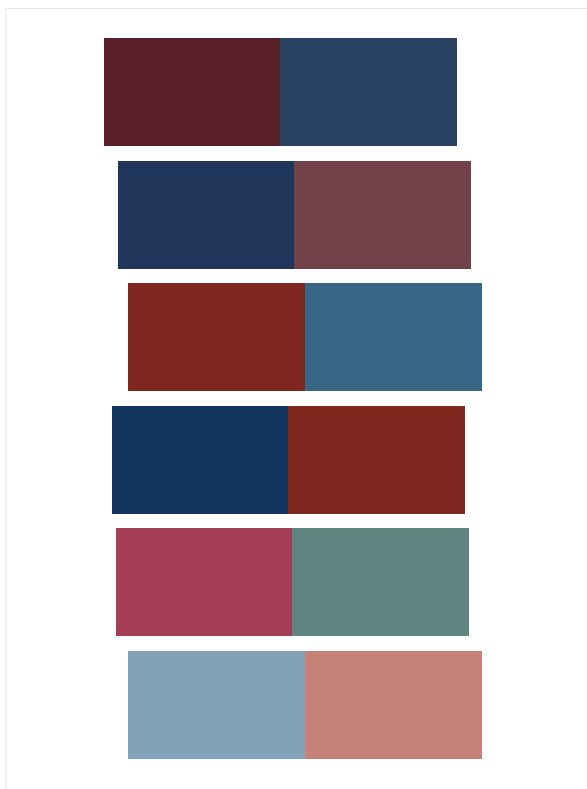
As "pure" colors were the first ink-based colors, uses of contrast of hue can be traced back to the early books of the middle ages:



Mondrian uses color hues, but with less saturation.

Cold / Warm Contrast

It's important to realize the dynamic nature of the cold/warm contrast. People tend to think that "red" is warm and "blue" is cold. As proved by Albers, here's an example of how extremely fragile that balance is.



[See example code](#)

Contrast of Extension

"He who first draws lines and then add color will never succeed in producing a clear, intense color effect ... Color have dimensions and directionality of their own" - Johannes Itten

Itten's contrast of extension relates to the fact that different colors do not have the same perceived volume. If a yellow rectangle and a purple rectangle are set side by side, the yellow rectangle appears much bigger.



This “contrast of extension” varies depending on the background color, neighbor colors, and arrangement.



[See example code](#)

Goethe and later Itten proposed the following extension values for colors:

- Yellow: 9
- Orange: 8
- Red: 6
- Violet: 3
- Blue: 4
- Green: 6

Here's those numbers visualized. Notice how balanced this color wheel is compared to Itten's normal color wheel with constant color sizes.

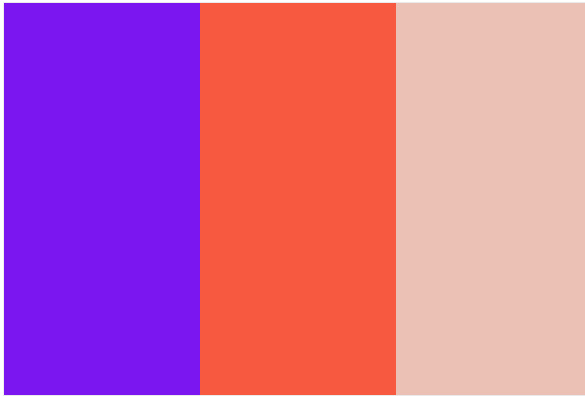


Of course this is a very basic treatment on the fact that colors have different volumes. It could be nice to have a constant to apply in code, but it varies greatly

with the background and matching colors. In the end it's a test for your visual perception.

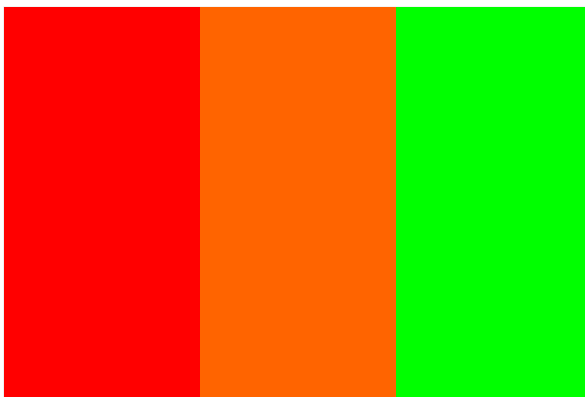
Generating colors

Here are two examples on how to generate random colors. First, here's a sketch with a `randomColor` function that generates a random color within certain HSV values.



[See example code](#)

If you don't want to generate completely random colors, you can put a number of colors in a JavaScript array, and generate a random number to select one of the colors in the array.

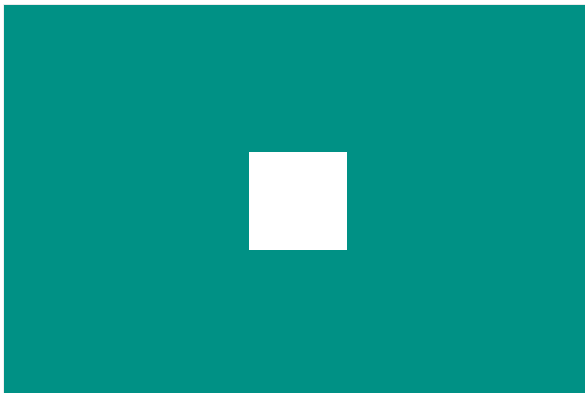


[See example code](#)

How to Find Opposite Brightness

For generative drawings we often want to [calculate the perceived brightness](#) of a color to be able to choose a font or shape color to draw on top of it. This is especially useful when working with random colors.

This is really easy to do in Rune.js, as you can just use the `light()` function to determine whether you should put black or white text on top of a specific color.



[See example code](#)